

TEORÍA DE BASE DE DATOS PROCESAMIENTO DE CONSULTAS

Prof. Sergio Salinas

TRABAJO PRÁCTICO

Preparación del entorno

Creación de tablas

```
CREATE TABLE student (  
  id INT,  
  name TEXT,  
  dept TEXT  
);
```

```
CREATE TABLE takes (  
  id INT,  
  course_id INT,  
  grade INT  
);
```

```
CREATE TABLE instructor (  
  id INT,  
  salary INT  
);
```

Carga de datos

```
INSERT INTO instructor  
SELECT i, (random()*100000)::INT  
FROM generate_series(1, 1000000) i;
```

1) Planes de ejecución

Consulta

```
EXPLAIN SELECT salary  
FROM instructor  
WHERE salary < 75000;
```

Con análisis

```
EXPLAIN ANALYZE  
SELECT salary  
FROM instructor  
WHERE salary < 75000;
```

2) Costos de consulta

```
EXPLAIN ANALYZE SELECT * FROM instructor;
```

```
EXPLAIN ANALYZE  
SELECT * FROM instructor WHERE salary < 1000;
```

Modificar parámetros

```
SET random_page_cost = 1;  
SET seq_page_cost = 10;
```

3) Selección sin índice

```
EXPLAIN ANALYZE  
SELECT * FROM instructor WHERE salary = 50000;
```

4) Selección con índice

```
CREATE INDEX idx_salary ON instructor(salary);
```

```
EXPLAIN ANALYZE  
SELECT * FROM instructor WHERE salary = 50000;
```

5) Clustering

```
CLUSTER instructor USING idx_salary;
```

6) Selecciones complejas

```
EXPLAIN ANALYZE  
SELECT * FROM instructor  
WHERE salary < 50000 AND id < 1000;
```

7) Sorting

```
EXPLAIN ANALYZE  
SELECT * FROM instructor ORDER BY salary;
```

```
SET work_mem = '1MB';  
SET work_mem = '100MB';
```

8) Nested Loop Join

```
EXPLAIN ANALYZE  
SELECT *  
FROM student s  
JOIN takes t ON s.id = t.id;
```

```
SET enable_hashjoin = off;  
SET enable_mergejoin = off;
```

9) Hash Join

```
SET enable_nestloop = off;  
  
EXPLAIN ANALYZE  
SELECT *  
FROM student s  
JOIN takes t ON s.id = t.id;
```

10) Merge Join

```
SET enable_hashjoin = off;  
SET enable_nestloop = off;  
  
CREATE INDEX ON student(id);  
CREATE INDEX ON takes(id);
```

11) Comparación de joins

Comparar los distintos algoritmos de join forzando cada uno y analizando su rendimiento.

12) Proyección y duplicados

```
EXPLAIN ANALYZE  
SELECT DISTINCT dept FROM student;
```

13) Pipelining

```
EXPLAIN ANALYZE  
SELECT *  
FROM student s  
JOIN takes t ON s.id = t.id  
WHERE t.grade > 7;
```

Investigar el uso de MATERIALIZED y el efecto sobre pipelining.

```
WITH takes_filtrada AS MATERIALIZED (  
SELECT id, course_id, grade  
FROM takes  
WHERE grade > 7  
)  
SELECT s.id, s.name, s.dept, t.course_id, t.grade  
FROM student s  
JOIN takes_filtrada t ON s.id = t.id;
```

14) Optimización

```
SELECT * FROM student WHERE id IN (  
    SELECT id FROM takes WHERE grade > 7  
);
```

```
SELECT *  
FROM student s  
JOIN takes t ON s.id = t.id  
WHERE t.grade > 7;
```